

Contents

guide 01 holistic_evaluation_of_language_models	1
0. 先给新手的结论	2
1. 当时的历史背景	2
2. 论文自己的结构地图	3
3. HELM 的核心对象: scenario, adaptation, metric	3
Scenario 是什么	4
Adaptation 是什么	4
Metric 是什么	5
4. HELM 和以前 benchmark 的本质区别	5
5. Core scenarios: 16 个核心场景	6
6. General metrics: 7 个维度怎么理解	6
Accuracy	6
Calibration	7
Robustness	7
Fairness	8
Bias and stereotypes	8
Toxicity	8
Efficiency	8
7. 模型选择和黑盒现实	8
8. Prompting adaptation: 标准化也会制造张力	9
9. 实验证据链: HELM 到底发现了什么	9
9.1 指标之间的关系	10
9.2 直接模型比较	10
9.3 Prompt 敏感性	10
9.4 Task-specific 结果	11
9.5 Targeted evaluations	11
10. 数学和张量级理解	11
11. 代码样例: mini HELM 结果矩阵	12
12. 代码样例: ECE 怎么算	13
13. 代码样例: robustness worst-case	13
14. 代码样例: 不要无脑平均	14
15. 和本仓库代码的对应关系	14
16. HELM 的局限	15
Results 的局限	15
Contamination 的局限	15
Adaptation 的局限	15
Measurement validity 的局限	16
Aggregation 的局限	16
17. HELM 对今天的意义	16
18. 新手常见误解	16
19. AI agent 正确学习这篇论文的方法	17
20. 读完必须能闭卷回答	18
21. 一页复盘	18

guide_01_holistic_evaluation_of_language_models

原论文: Holistic Evaluation of Language Models

本地原文 PDF: learning/eval-foundations/paper/01_helm.pdf

作者: Percy Liang, Rishi Bommasani, Tony Lee, et al., Stanford CRFM

年份: 2022 arXiv, 2023 TMLR version

读法定位: 这不是一篇提出新模型的论文, 而是一篇给 LLM 建立“评测语法”的系统论文。读懂它以后, 你看任何榜单、leaderboard、模型报告、安全报告, 都应该能追问: 它测了什么 scenario, 用了什么 adaptation, 算了哪些 metric, 没有测什么, 结论能不能支持模型选择。

0. 先给新手的结论

HELM 的核心贡献可以压成一句话:

不要用一个平均分定义语言模型, 要用一个结构化矩阵描述模型。

这个矩阵大致长这样:

```
result[model, scenario, metric]
```

```
model      = GPT-NeoX / OPT / BLOOM / davinci / text-davinci-002 / ...
scenario    = NaturalQuestions / MMLU / IMDB / XSUM / CivilComments / ...
metric      = accuracy / calibration / robustness / fairness / bias /
              toxicity / efficiency
```

过去的很多 benchmark 更像“考试合集”: 模型在几个数据集上拿 accuracy, 最后平均一下。HELM 说这不够。语言模型已经是 foundation model, 能被用到问答、摘要、搜索、内容审核、推理、写作、安全等场景。一个模型可能准确率高, 但校准差; 可能问答强, 但毒性检测弱; 可能平均分高, 但对某些群体输入掉分; 可能小模型慢不了多少但部署成本低。单分数会把这些 trade-off 都藏掉。

所以 HELM 做三件事:

1. 先建立 taxonomy: 明确 scenario 空间和 metric 空间。
2. 再选一个可实现子集: 16 个 core scenarios, 7 类 metrics。
3. 用标准化 prompting 去评 30 个模型, 并公开 prompt、completion 和结果。

论文摘要里有几个数字必须记住:

- 16 个 core scenarios。
- 7 类 general metrics。
- 112 个 scenario-metric pair 中实际测了 98 个, 也就是 87.5%。
- 7 组 targeted evaluations, 覆盖 26 个 targeted scenarios。
- 30 个 prominent language models。
- 总共 42 个 scenarios, 其中 21 个之前不是主流 LM evaluation 常用场景。
- 之前模型平均只覆盖 HELM core scenarios 的 17.9%; HELM 把覆盖提升到 96.0%。

这些数字不是装饰。它们说明 HELM 的野心不是“多跑几个数据集”, 而是把评测从 leaderboard 变成一张可审计的地图。

1. 当时的历史背景

2020 到 2022 年, 语言模型进入 GPT-3 之后的阶段。大家已经知道 scale 很重要, few-shot prompting 很惊艳, 开放模型和 API 模型都在快速出现。但评测生态还很碎:

- SQuAD、SuperGLUE、MMLU 这类 benchmark 主要强调 accuracy。
- BIG-Bench、lm-evaluation-harness 把任务集合变大, 但仍常被读成“更多题目的准确率”。

- toxicity、bias、fairness、calibration、robustness 往往是单独论文或单独数据集。
- 不同模型论文选择不同 benchmark，很难横向比较。
- 商业 API、closed model、limited-access model 让研究者无法访问训练数据和内部激活。

这带来一个现实问题：如果你要选一个模型进产品，平均准确率不是充分信息。

例如：

- 客服分类需要稳定、低延迟、可解释的错误模式。
- 医疗/法律场景需要校准和拒答能力。
- 内容审核场景需要关注 subgroup disparity。
- 新闻摘要需要 faithfulness，不只是 ROUGE。
- 面向公众的系统需要毒性、偏见、隐私和版权风险。

HELM 的背景判断是：benchmark 会“定方向”。如果社区只优化一个分数，模型研发就会被这个分数牵着走。于是作者把 evaluation 当成一种公共基础设施，而不仅是实验章节里的表格。

2. 论文自己的结构地图

读原文时建议按这个顺序走：

1. Abstract 和 Introduction: 抓住 holistic evaluation 的三个元素。
2. Section 2 Preliminaries: 记住 scenario、adaptation、metric 三元组。
3. Section 3 Core scenarios: 看 scenario taxonomy 怎么拆成 task、domain、language。
4. Section 4 General metrics: 看 7 类 metric 怎么选，为什么只能测黑盒可测的 desiderata。
5. Section 6 Models: 看 30 个模型、access 条件、成本、污染和 live system 问题。
6. Section 7 Adaptation via prompting: 看 5-shot prompt 标准化的细节。
7. Section 8 Experiments: 不要只看谁第一，要看指标关系、prompt 敏感性和反直觉案例。
8. Section 10 Missing 和 Section 11 Limitations: 看 HELM 自己承认测不到什么。

论文里的关键图表：

- Figure 2/8: scenario taxonomy，不是随便堆 dataset。
- Figure 3/Table 4: 多 metric 矩阵，16 x 7 的核心。
- Figure 5: evaluation run 的三个组件。
- Figure 7/23: prompt adaptation 的具体格式。
- Figure 17: ECE calibration 的计算直觉。
- Figure 18/19: robustness/fairness perturbation。
- Figure 24/25: accuracy 和其他指标的关系。
- Figure 31/32/33: prompt seed、shot count、multiple-choice adaptation 的敏感性。
- Table 5/6: 30 个模型与评测成本。
- Table 8: disinformation 的人工评测。

3. HELM 的核心对象: scenario, adaptation, metric

HELM 的最小评测单元不是“数据集”，而是一次 run：

```
evaluation_run =
    scenario
    + adaptation_method
    + metric
```

三者分别回答三个问题：

scenario: what do we want the model to do?
adaptation: how do we turn the raw LM into a task system?
metric: how good are the resulting predictions?

Scenario 是什么

Scenario 实例化一个语言模型用例。论文把 scenario operationalize 成一组 instances。每个 instance 至少包含:

```
instance = {  
  input: string,  
  references: [  
    {text: string, properties: {...}},  
    ...  
  ]  
}
```

对于 multiple choice:

```
input:  
  Question: Which term describes the body's ability to maintain its normal state?  
  A. Anabolism  
  B. Catabolism  
  C. Tolerance  
  D. Homeostasis
```

```
references:  
  D is correct
```

Scenario 又可以拆成:

```
scenario = (task, domain, language)
```

```
task:  
  question answering / summarization / sentiment / retrieval / ...
```

```
domain:  
  what: topic or genre, such as Wikipedia, news, movie review  
  who: speaker or subject group  
  when: time and circumstances
```

```
language:  
  English, English varieties, future multilingual coverage
```

这套拆法的设计理由是: 评测不应该只说“我用了 NaturalQuestions”, 而应该能说清楚它覆盖了什么 use case, 没覆盖什么 use case。

Adaptation 是什么

语言模型本身只是一个 text-in, text-out 的黑盒:

```
LM(prompt, decoding_params)  
-> completion
```

-> log probabilities

Adaptation 是把原始 LM 变成任务系统的过程。它可以是 prompting、lightweight fine-tuning、full fine-tuning。HELM 为了横向比较和适配 API 黑盒，主要用 prompting，而且默认使用 5-shot prompting。

关键点: HELM 不假设能看模型内部激活，也不假设能看训练数据。这是现实 API 研究环境下的妥协。

Metric 是什么

Metric 把 completion 和 logprob 变成分数。Accuracy 只是其中一种。HELM 强调 metric 是 desiderata 的具体操作化: 我们希望系统有准确性、校准、鲁棒性、公平性、低偏见、低毒性、效率，于是要为这些抽象要求设计可计算指标。

一个 evaluation run 的流程可以画成:

```
scenario instances
  |
  v
prompt formatter + k-shot examples
  |
  v
model API / local LM
  |
  v
completion + logprobs
  |
  v
metric functions
  |
  v
score cell: R[model, scenario, metric]
```

4. HELM 和以前 benchmark 的本质区别

以前很多 benchmark 是这样的:

```
dataset_1 -> accuracy
dataset_2 -> accuracy
dataset_3 -> accuracy
average score
```

HELM 是这样的:

```
for each model:
  for each scenario:
    for each metric that is valid:
      run standardized adaptation
      record score and raw predictions
```

它的核心不是“更多数据集”，而是“评测设计可解释”：

- 先列出抽象 taxonomy。
- 再说明为什么选这些 scenario。

- 再说明哪些 metric 可测，哪些暂时不可测。
- 再把缺口显式写出来。

这就是论文反复讲的 recognition of incompleteness。HELM 的好处不是它完整，而是它把“不完整在哪里”暴露出来。

5. Core scenarios: 16 个核心场景

HELM core scenarios 覆盖 6 类 user-facing tasks:

1. Question answering: NaturalQuestions open-book, NaturalQuestions closed-book, NarrativeQA, QuAC, BoolQ, HellaSwag, OpenBookQA, TruthfulQA, MMLU。
2. Information retrieval: MS MARCO regular, MS MARCO TREC。
3. Summarization: CNN/DailyMail, XSUM。
4. Sentiment analysis: IMDB。
5. Toxicity detection: CivilComments。
6. Miscellaneous text classification: RAFT。

选择原则有三个:

- coverage: 覆盖 task/domain/language 空间。
- minimality: 不无限堆数据集，尽量少而代表性强。
- user-facing: 优先实际应用场景。

但作者也很诚实地说，这些选择受 feasibility 约束。比如 HELM 当时主要是 English，许多行业域、非英语语言、方言、历史文本、新兴创作任务都缺。

这点对你学习很重要: benchmark 的数据集选择本身就是价值判断。看榜单时不要问“它准不准”，要问“它的 scenario selection 是否匹配我的使用场景”。

6. General metrics: 7 个维度怎么理解

HELM 的 7 类 general metrics 是:

1. accuracy
2. calibration and uncertainty
3. robustness
4. fairness
5. bias and stereotypes
6. toxicity
7. efficiency

论文 Table 4 展示 16 个 core scenarios x 7 个 metric categories 的矩阵。理论上是 112 个 pair，实际测了 98 个。没测的原因通常是:

- metric 对该 scenario 不成立，比如没有 generation 就很难测生成毒性。
- 测量 validity 可疑，比如长文本摘要上的某些 perturbation 不可靠。

Accuracy

Accuracy 是 umbrella term，不同 scenario 对应不同主分数:

```
text classification -> exact match
question answering  -> F1 or exact match
retrieval            -> RR@10, NDCG@10
summarization        -> ROUGE, BERTScore, faithfulness metrics
```

新手容易误解: accuracy 是平均在 test instances 上的。平均高不等于每个 subgroup 高, 也不等于真实部署风险低。

Calibration

Calibration 问的是: 模型说自己有多确定, 这个概率可信吗?

如果模型对 1000 个样本都给 0.7 置信度, 那么一个 calibrated model 应该大约答对 700 个。

HELM 用 ECE, expected calibration error。简化公式:

Given N examples, split predictions into B bins by confidence.

```
ECE =
  sum over bins b:
    (count(b) / N) * abs(accuracy(b) - confidence(b))
```

其中:

- accuracy(b) 是 bin 里实际正确率。
- confidence(b) 是 bin 里平均预测概率。
- ECE 越小越好。

HELM 还测 selective classification:

```
sort examples by model confidence
keep top C fraction
measure accuracy on kept examples
```

这回答另一个问题: 模型即使概率数值不准, 能不能知道哪些题更容易。

Robustness

Robustness 问的是: 输入稍微变化, 模型会不会崩?

HELM 主要测 local robustness, 包括:

- invariance: 语义不变扰动, 比如 typo、大小写、轻微改写, 输出应该不变。
- equivariance: 语义改变扰动, 比如 contrast set, 输出应该跟着变。

一个简化的 worst-case score:

```
for each original example i:
  create perturbations T_i = {x_i_1, x_i_2, ...}
  score_i = min over x in T_i of correct(model(x), y_i)
```

```
robustness = mean_i score_i
```

设计理由: 部署时用户输入不会总是干净 benchmark 句子。只在原始测试集上准确, 不代表真实世界稳定。

Fairness

HELM 的 fairness 测法包含 perturbation-based fairness 和 subgroup performance。例子:

- 把输入里的 gendered terms 替换, 看预测是否不合理变化。
- 对 dialect perturbation 看性能是否掉。
- 在有 demographic metadata 的数据上比较不同 subgroup accuracy。

一个简单 group gap:

```
group_accuracy[g] = correct examples in group g / total examples in group g
fairness_gap = max_g group_accuracy[g] - min_g group_accuracy[g]
```

但论文很谨慎: fairness 是复杂社会概念, 不是一个公式能定义完。HELM 的做法是 scalable proxy, 能扩大覆盖, 但牺牲了一些社会语境的细腻度。

Bias and stereotypes

Bias 主要看 generation 中的 demographic representation 和 stereotypical association。它和 fairness 不一样:

- fairness 更常看任务表现是否因群体扰动掉分。
- bias 更常看模型生成内容里是否复制刻板关联。

这就是为什么论文会发现: fairness 和 accuracy 在它的测法下强相关, 但 generation bias 不一定跟 accuracy 同向。

Toxicity

Toxicity 看模型输出是否有毒性内容。HELM 用 PerspectiveAPI 等自动工具, 但也明确指出这类工具有局限:

- 不同群体对 toxic 的判断可能不同。
- 自动 toxicity detector 自己可能有偏差。
- 低频毒性仍可能造成高伤害。

所以 toxic rate 不能被读成“安全证书”。

Efficiency

Efficiency 不是附属指标。模型部署要看:

- inference runtime。
- token cost。
- query cost。
- GPU hours。
- model access 条件。

Table 5 里作者记录了每个模型的 total tokens、total queries 和评测成本。例如商业 API 用美元成本, open model 用 GPU hours。这个设计让 evaluation 不只是学术分数, 也接近真实工程代价。

7. 模型选择和黑盒现实

HELM 评了 30 个模型, 覆盖:

- open weight models: GPT-J, GPT-NeoX, OPT, BLOOM, T5, T0+ +, UL2 等。
- limited-access APIs: OpenAI davinci/text-davinci, AI21 Jurassic, Cohere 等。
- closed but research-access models: Anthropic-LM, TNLG v2。

论文很强调 access condition, 因为这影响可复现性和公平性。

几个现实限制:

1. 有些模型不给 logprobs, 所以需要概率的 scenario 或 metric 无法评。
2. 有些 API live system 会更新, 结果只对应当时查询的版本。
3. 模型训练数据不透明, contamination 无法彻底排除。
4. 同样的 5-shot prompt 对某些模型公平, 对另一些模型可能不公平。

这段对现在尤其重要。很多 leaderboard 看似客观, 其实背后有版本、prompt、access、污染、成本的混杂因素。HELM 的价值之一就是把这些工程事实写进 evaluation design。

8. Prompting adaptation: 标准化也会制造张力

HELM 默认用 5-shot prompting, 把 train examples 放进 prompt, 然后让模型回答 test instance。

prompt 大致是:

```
{instructions}

{train input}
{train references}
{train output}

... repeated up to 5 shots ...

{test input}
{test references}
{test output prefix}
```

decoding 参数也要固定, 例如 temperature、max tokens、stop sequences、num runs、max evaluation instances。

为什么这复杂? 因为 prompt 不是无关细节。论文 Section 8.2 专门证明:

- in-context examples 的选择会影响结果。
- shot 数从 0 到 1 经常显著提升, 但更多 shot 不一定稳定提升。
- prompt formatting 对不同模型影响巨大。
- multiple choice 的 joint / separate / separate-calibrated 适配方式能大幅改变 accuracy。

一个非常关键的例子: HellaSwag 上 OPT 175B 使用 separate 方法能到 79.1%, joint 方法会掉到 30.2%。这说明“同一个数据集、同一个模型”也会因为 adaptation method 得到完全不同的结论。

所以 HELM 的标准化不是终点, 而是让比较可审计的起点。读榜单时要问:

This score is not just model ability.

This score = model + prompt format + decoding + metric + dataset sample.

9. 实验证据链: HELM 到底发现了什么

HELM 的实验不是为了宣布唯一冠军, 而是为了暴露模型在不同维度的结构性差异。

9.1 指标之间的关系

Figure 24/25 研究 accuracy 和其他 metric 的关系。

主要发现:

- accuracy、robustness、fairness 在 HELM 的测量方式下高度相关。
- calibration 和 accuracy 的关系非常 scenario-dependent。
- toxicity rate 在 core scenarios 中通常很低，但不能说明所有场景都低风险。
- bias 和 toxicity 的关系不稳定，减少一种 harm 可能不等于减少另一种。
- efficiency 和 accuracy 没有统一强 trade-off，很多时候取决于 scenario 输入输出长度和系统实现。

这对学习者的意义是: 不要把“模型更强”理解成所有维度同时更好。你要学会看 metric pair。

9.2 直接模型比较

Figure 26 用 head-to-head win rate 比较模型。text-davinci-002 是整体最准确的模型，accuracy head-to-head win rate 超过 90%。TNLG v2 530B 和 Anthropic-LM v4-s3 52B 也很强。

但论文给出的解读很细:

- scale 在同一模型家族内通常和 accuracy 单调相关。
- 跨家族时，scale 不是唯一解释；instruction tuning/human feedback 很重要。
- top accuracy 模型通常至少 50B 参数，但更大不总是更有效。
- bias/toxicity 排名和 accuracy 排名很不一样。

这说明现代模型能力来自 scale、data、instruction tuning、RLHF、prompt compatibility 等多个因素，不是只看参数量。

9.3 Prompt 敏感性

HELM 对 prompt 做了几类分析。

in-context example seed:

- 大多数模型在大多数 scenario 上 seed 方差不大。
- 但 NaturalQuestions open-book 等场景会很敏感。

shot count:

- 从 0-shot 到 1-shot 往往提升明显。
- 继续加到 2/4/8/16 不一定统一提升。
- CNN/DailyMail 这类摘要场景可能 0-shot 更好，因为 few-shot reference length 反而误导模型。

prompt formatting:

- 最优 prompt 不同模型不同。
- 某些 prompt 可以让一个模型大幅提高，同时让另一个模型崩掉。

multiple choice adaptation:

- joint: 一次把所有选项给模型，让它生成答案。
- separate: 每个选项单独 query，用 logprob 排序。
- separate-calibrated: 再用选项先验概率做校准。

HellaSwag 中 separate 明显优于 joint，但 Anthropic-LM 在一些场景上偏好 joint。这提出一个深问题: 对所有模型用同一 adaptation 是否公平？如果每个模型用自己最优 adaptation，又是否可比？

9.4 Task-specific 结果

几个值得记的例子:

- QA: text-davinci-002 在 9 个 QA scenarios 上都是最准确, 但第二名随场景变化。
- TruthfulQA: text-davinci-002 大幅领先, 论文报告 62.0% vs Anthropic-LM 约 35%。
- IR: prompt-based LM reranker 在 MS MARCO 上有竞争力, 但效率远不如专门 retrieval system。
- Summarization: ROUGE 分数和人的质量感受不总一致, length control 是关键问题。
- IMDB sentiment: 很多模型超过 90%, 但 contrast set 会暴露更大 robustness drop。
- CivilComments toxicity detection: 很多模型只略高于 chance, 且 subgroup performance 差异明显。
- RAFT: 长尾分类很能区分模型, text-davinci-002 这种总体强模型也可能在某些子任务弱。

这些结果教你一个读论文方法: 先看 macro trend, 再找 counterexample。真正有用的洞察通常藏在“总体强但这里弱”、“平均高但 subgroup 掉”、“准确高但校准差”这些地方。

9.5 Targeted evaluations

HELM 还做 targeted evaluations, 专门测:

- language: The Pile, TwitterAAE, ICE, BLiMP。
- knowledge: MMLU, NaturalQuestions, WikiFact 等。
- reasoning: synthetic reasoning, GSM8K, MATH, LSAT, HumanEval, APPS 等。
- memorization and copyright。
- disinformation。
- social bias: BBQ。
- toxicity generation: RealToxicityPrompts, BOLD。

关键发现:

- TwitterAAE 上 African American English subset 的 BPB 明显更差, 说明语言模型也有方言/群体差异。
- code-davinci-002 在 reasoning 场景特别强, 包括一些自然语言推理任务。
- copyrighted text regurgitation 平均不常见, 但热门书籍会出现非平凡长段复现风险。
- BBQ 中高 accuracy 模型在 ambiguous contexts 上可能更显著复制社会偏见。
- toxic prompts 会显著提高 toxic generation rate; non-toxic prompts 下 toxic rate 低很多。
- disinformation 人工评测显示模型能生成支持给定 thesis 且像新闻标题的文本, 应被视为风险下界。

10. 数学和张量级理解

HELM 不是神经网络架构论文, 所以没有 attention tensor 那种 shape。但它有自己的“评测张量”。

最重要的张量是:

$R[M, S, K]$

M = number of models

S = number of scenarios

K = number of metrics

其中一个 cell 是:

$R[m, s, k] =$
metric_k(

```

    outputs = adapt_and_run(model_m, scenario_s),
    references = scenario_s.references
)

```

如果 metric 需要 subgroup:

```
R_group[m, s, k, g]
```

g = demographic group / dialect / subject / category

如果 metric 需要 perturbation:

```
X_perturbed[i, t]
```

i = original instance index

t = perturbation index

如果 metric 需要 prompt seed:

```
R_seed[m, s, k, r]
```

r = random seed / in-context example set

所以 HELM 的“张量级图示”可以这样画:

```

instances[s]
-> prompts[s, r, i]
-> model outputs[m, s, r, i]
-> per-example scores[m, s, r, i, k]
-> aggregate R[m, s, k]
-> analysis over metrics, groups, seeds, time, access, scale

```

这就是为什么 HELM 比单 benchmark 难读: 它不是一条 loss 曲线, 而是一个高维测量系统。

11. 代码样例: mini HELM 结果矩阵

本仓库的 `learning/eval-foundations/src/helm_local.py` 做了教学版 micro-HELM。它没有真实 HELM 那么大, 只保留核心思想:

- 多个 scenario。
- 每个 scenario 一个 metric。
- 同一个 model 跑出一个 score matrix。

你可以把核心结构理解成:

```

from dataclasses import dataclass

@dataclass
class Cell:
    scenario: str
    metric: str
    value: float

def run_micro_helm(model):
    cells = []
    cells.append(Cell("knowledge", "exact_match", eval_qa(model)))

```

```

cells.append(Cell("reasoning", "exact_match", eval_reasoning(model)))
cells.append(Cell("summarization", "rouge1_proxy", eval_summ(model)))
cells.append(Cell("robustness", "exact_match", eval_robust(model)))
return cells

```

注意这个结构的关键不是 evaluator 多复杂，而是输出不再是一个数字：

```

model -> [
    (knowledge, exact_match, 0.67),
    (reasoning, exact_match, 0.50),
    (summarization, rouge1_proxy, 0.80),
    (robustness, exact_match, 0.00),
]

```

一旦你把结果保存成 matrix，就可以问：

- 哪个 scenario 掉分？
- 哪个 metric 和 accuracy 不一致？
- 哪个模型适合哪个应用？
- 是否存在 Pareto dominance？

12. 代码样例: ECE 怎么算

HELM calibration 的核心可以用下面的 toy 代码理解：

```

def expected_calibration_error(confidences, correct, n_bins=10):
    pairs = sorted(zip(confidences, correct), key=lambda x: x[0])
    n = len(pairs)
    if n == 0:
        return 0.0

    ece = 0.0
    for b in range(n_bins):
        lo = b * n // n_bins
        hi = (b + 1) * n // n_bins
        bucket = pairs[lo:hi]
        if not bucket:
            continue
        avg_conf = sum(c for c, _ in bucket) / len(bucket)
        avg_acc = sum(1 for _, ok in bucket if ok) / len(bucket)
        ece += (len(bucket) / n) * abs(avg_acc - avg_conf)
    return ece

```

如果一个模型 accuracy 高但 ECE 大，意思是它答对不少，但“知道自己什么时候对”这件事做得差。对于高风险应用，这可能比少几个 accuracy point 更重要。

13. 代码样例: robustness worst-case

robustness 的思想是：对同一个样本造几个扰动，只要其中一个扰动下答错，就说明这一点不稳。

```

def robust_accuracy(model, examples, perturb):
    total = 0
    robust = 0

```

```

for ex in examples:
    total += 1
    variants = perturb(ex["input"])
    ok_all = True

    for x in variants:
        pred = model(x)
        if normalize(pred) != normalize(ex["gold"]):
            ok_all = False
            break

    if ok_all:
        robust += 1

return robust / max(1, total)

```

这个 toy 函数对应论文里 Figure 18 的直觉。真实 HELM 还会区分 invariance 和 equivariance，并且 perturbation 的有效性本身需要审慎验证。

14. 代码样例: 不要无脑平均

HELM 不给一个 universal aggregation，因为不同应用权重不同。但你本地可以做一个显式加权，训练自己“声明价值判断”。

```

def weighted_score(cells, weights):
    score = 0.0
    used = 0.0
    for c in cells:
        key = (c.scenario, c.metric)
        w = weights.get(key, 0.0)
        score += w * c.value
        used += w
    return score / used if used else 0.0

weights_for_mobile_app = {
    ("knowledge", "exact_match"): 0.25,
    ("reasoning", "exact_match"): 0.20,
    ("summarization", "rouge1_proxy"): 0.15,
    ("robustness", "exact_match"): 0.40,
}

```

这段代码的学习价值是：你必须承认 aggregation 是选择，不是自然真理。HELM 的设计让选择暴露出来。

15. 和本仓库代码的对应关系

推荐按这个顺序读：

1. [learning/eval-foundations/lectures/04-helm.md](#) 先看本专题对 HELM 的压缩讲解。
2. [learning/eval-foundations/src/common.py](#) 看 EvalSample, EvalResult, make_mock_model,

`format_multiple_choice`, `accuracy`, `group_accuracy`。这对应 HELM 的 `instance`、`adaptation`、`metric` 基础对象。

3. `learning/eval-foundations/src/helm_local.py` 看 `run_helm_local` 和 `HelmCell`。这对应 HELM 的 `scenario-metric matrix`。
4. `learning/eval-foundations/src/eval_pipeline.py` 看多个 benchmark 如何联跑并生成 `mark-down report`。这对应 HELM 的 `benchmark infrastructure`。
5. `learning/eval-foundations/src/contamination_check.py` 看污染检测的 toy 版本。对应论文里 `model contamination` 的局限讨论。
6. `learning/eval-foundations/src/lm_eval_adapter.py` 看如何把 `task/result` 抽象成类似 `lm-eval-harness` 的格式。对应 HELM 和其他 eval infrastructure 的关系。

本地实验:

```
python learning/eval-foundations/src/tests/test_eval.py
python learning/eval-foundations/src/helm_local.py
python learning/eval-foundations/src/eval_pipeline.py
```

如果你想做 30-60 分钟实验, 建议改 `helm_local.py`:

1. 给 `robustness scenario` 增加 3 个 `typo/case variants`。
2. 写一个 `second mock model`, 让它 `knowledge` 强但 `robustness` 弱。
3. 跑 `run_helm_local` 对比两个模型。
4. 不要算一个平均分, 分别解释每个 `cell`。
5. 写下哪个模型适合部署到“用户输入很脏”的场景。

16. HELM 的局限

HELM 论文自己很清醒, 局限至少有五类。

Results 的局限

HELM 结果不等于所有实际应用结论。实际系统可能会:

- `fine-tune`。
- `use retrieval`。
- `use tool`。
- `use domain data`。
- `use human review`。
- `use safety filter`。

所以 HELM 分数只能作为通用透明度信息, 不能替代应用侧评测。

Contamination 的局限

模型训练数据巨大且不完全公开。测试集可能出现在训练数据里, 尤其 `few-shot evaluation` 更容易被污染影响。HELM 只能记录已知污染, 无法彻底证明无污染。

Adaptation 的局限

HELM 使用 `prompting`。换成 `fine-tuning`、`prompt optimization`、`chain-of-thought`、`tool use`, 结果可能不同。Section 8.2 已经证明 `prompt` 细节会显著影响模型表现。

Measurement validity 的局限

很多 metric 是社会概念的 proxy。比如 fairness、toxicity、bias 的测量依赖 perturbation 或自动 classifier，未必能完全代表真实社会伤害。

Aggregation 的局限

HELM 产生 score matrix，不提供唯一总分。这既是优点也是成本。复杂结果更真实，但也更难消费。用户可能仍想要一个数字，但任何单数字都隐含价值权重。

17. HELM 对今天的意义

HELM 对今天 LLM 学习仍然重要，原因不是它的模型列表最新，而是它提供了读评测的框架。

现在你看任何模型报告，都应该按 HELM 的方式问：

1. scenarios:
这些任务是否覆盖我的真实 use case?
2. metrics:
除了 accuracy, 有没有 calibration, robustness, safety, efficiency?
3. adaptation:
prompt 是否固定? 是否用了 CoT? 是否给每个模型调过 prompt?
4. model access:
是 open weight, API, closed, live system 还是版本锁定?
5. raw outputs:
有没有公开 prompts 和 completions?
6. incompleteness:
作者有没有明确说没测什么?
7. aggregation:
总分权重是否合理? 是否隐藏 trade-off?

对 AI agent 学习尤其关键。Agent 的表现更容易被 prompt、工具、memory、retrieval、judge、任务拆解方式影响。如果只看一个 pass rate，就会错过很多失败模式。HELM 教你把 agent evaluation 拆成矩阵：

```
agent_result[agent, task_family, metric, tool_condition, seed]
```

这比“某 agent 在某榜单 70%”更有学习价值。

18. 新手常见误解

误解 1: HELM 是一个更大的榜单。

更准确: HELM 是一个评测设计框架，榜单只是输出之一。

误解 2: metric 越多越客观。

更准确: metric 越多，价值冲突越透明。但每个 metric 仍有测量假设。

误解 3: 标准化 prompt 就公平。

更准确: 标准化让比较可复现, 但不同模型可能适合不同 adaptation。公平本身仍有争议。

误解 4: toxicity rate 低说明模型安全。

更准确: 它只说明在这些 prompts、这个 detector、这个 decoding 下毒性低。

误解 5: 不能聚合就没法决策。

更准确: 可以聚合, 但权重要由应用需求显式决定, 不能伪装成 universal truth。

19. AI agent 正确学习这篇论文的方法

第一轮: 让 agent 帮你审题, 而不是总结。

我在读 HELM。请你只问我问题, 不要给答案。

先考我 scenario, adaptation, metric 三个概念。

每次只问一个问题, 我答完后指出哪里不精确。

第二轮: 让 agent 陪你画矩阵。

请让我闭卷画 HELM 的 result tensor。

我会写 `R[model, scenario, metric]`。

你检查我是否漏掉 prompt seed, perturbation, subgroup, raw output。

第三轮: 让 agent 陪你读实验。

请按 Figure 24/25 的思路考我:

accuracy 和 calibration/robustness/fairness/bias/toxicity/efficiency
分别是什么关系?

只问关系和可能原因, 不要让我背图号。

第四轮: 跑本地 toy code。

```
python learning/eval-foundations/src/tests/test_eval.py
```

```
python learning/eval-foundations/src/helm_local.py
```

```
python learning/eval-foundations/src/eval_pipeline.py
```

跑完你要自己写:

1. 本地 toy HELM 有几个 scenario?
2. 每个 scenario 的 metric 是什么?
3. 为什么一个平均分不够?
4. 如何构造一个 knowledge 强但 robustness 弱的 mock model?
5. 这个 toy 和真实 HELM 差在哪里?

第五轮: 闭卷输出。

请你按 10 分制给我评分。

我会用 300 字闭卷解释 HELM。

评分维度:

scenario/adaptation/metric 各 2 分,

multi-metric evidence 2 分,

limitations 2 分。

只指出缺口, 不要替我重写。

20. 读完必须能闭卷回答

1. HELM 为什么反对只用一个平均分评价语言模型?
2. Holistic evaluation 的三个元素是什么?
3. Scenario、adaptation、metric 分别回答什么问题?
4. Scenario 为什么拆成 task、domain、language?
5. HELM core scenarios 有哪些大类?
6. 7 类 general metrics 分别是什么?
7. Table 4 的 98/112 和 87.5% 说明什么?
8. ECE 怎么计算? 它和 accuracy 有什么区别?
9. Robustness 的 invariance 和 equivariance 有什么区别?
10. HELM 的 fairness 测法为什么只是 proxy?
11. 为什么 prompt formatting 会影响模型比较的公平性?
12. HellaSwag 上 joint/separate multiple choice 的例子说明什么?
13. text-davinci-002、TNLG v2、Anthropic-LM 的结果分别说明 scale 和 instruction tuning 什么关系?
14. 为什么 contamination 是 HELM 无法彻底解决的问题?
15. HELM 为什么不提供 universal single score?
16. 如果你要给 agent 系统做 HELM-style eval, 你会怎么定义 result tensor?

21. 一页复盘

HELM 是现代语言模型评测基础设施论文。它的中心观点是: 语言模型不能被单一 accuracy 或平均榜单定义, 应该被放进一个 model x scenario x metric 的结果矩阵中。论文先提出 scenario 和 metric taxonomy, 再选择 16 个 core scenarios 和 7 类 metrics, 在 112 个理论 pair 中测了 98 个, 并用标准化 5-shot prompting 评估 30 个模型和 42 个 scenarios。它把 evaluation run 拆成 scenario、adaptation、metric 三元组, 强调 prompt、decoding、logprob、raw completion、subgroup、perturbation、efficiency 都是结果的一部分。实验发现 accuracy、robustness、fairness 在其测法下强相关, 但 calibration、bias、toxicity、efficiency 不会简单跟随 accuracy; prompt seed、shot count、prompt format、multiple-choice adaptation 都能显著改变结论。HELM 的价值不是给出永恒排名, 而是教你读模型证据链: 看覆盖、看指标、看 trade-off、看 raw outputs、看缺失、看污染、看 aggregation 权重。它也提醒你, 评测本身是一种价值选择和工程系统。